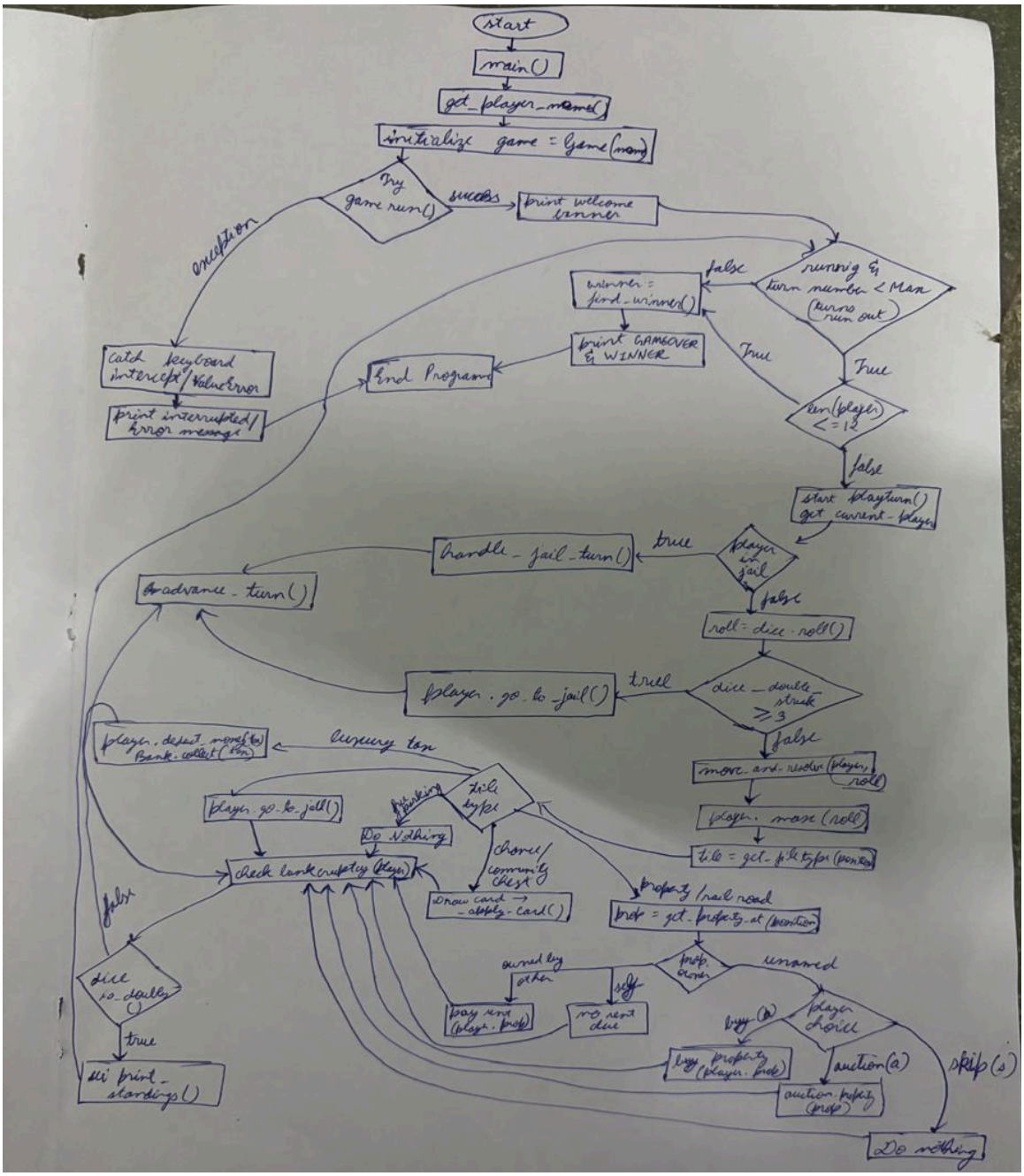
Samraddhi Singhai 2024115015

Question 1 Report

1. Overview

This report documents the quality assurance work completed for version 1.2 of MoneyPoly. Testing was carried out using the PyLint static analysis tool on a module-by-module basis. **Following static analysis, the codebase structure was reorganised so that main.py resides in the same directory as all other modules, and the updated version has been submitted on Moodle.**



2. Static Analysis — Module-by-Module Fixes

2.1 bank.py

Errors raised by PyLint:

bank.py:1:0: C0114: Missing module docstring | bank.py:2:0: E0401: Unable to import 'moneypoly.config' | bank.py:5:0: C0115: Missing class docstring | bank.py:1:0: W0611: Unused import math

Iteration	Fix Applied
1	Added module docstring: "Defining the Bank class, which manages the game's finances and loan system." Removed unused import math.
2	Added class docstring to Bank: "Manages the bank's cash, loans, and payouts."
3	Fixed import error — changed 'from moneypoly.config import BANK_STARTING_FUNDS' to 'from config import BANK_STARTING_FUNDS'.

2.2 board.py

Errors raised by PyLint:

board.py:1:0: C0114: Missing module docstring | board.py:1:0: E0401: Unable to import 'moneypoly.property' | board.py:2:0: E0401: Unable to import 'moneypoly.config' | board.py:108:11: C0121: Comparison 'prop.is_mortgaged == True' should be 'prop.is_mortgaged is True'

Iteration	Fix Applied
4	Fixed import — changed 'from moneypoly.property import Property, PropertyGroup' to 'from property import Property, PropertyGroup'.
5	Fixed import — changed 'from moneypoly.config import' to 'from config import'.
6	Added module docstring: "Represents the game board, tiles, and properties."
7	Changed == to is: 'if prop.is_mortgaged == True' changed to 'if prop.is_mortgaged is True' (singleton comparison).

2.3 cards.py

Errors raised by PyLint:

cards.py:1:0: C0114: Missing module docstring | cards.py:4:0: C0301: Line too long (111/100) | cards.py:5:0: C0301: Line too long (112/100) (for lines 4–30) | cards.py:61:46: C0303: Trailing whitespace

Iteration	Fix Applied
8	Added module docstring: "Contains Chance and Community Chest card decks".
9	Reformatted card definitions from single long lines to multi-line dictionary format to stay within 100 characters per line.
10	Accidentally left whitespace on line 61 — got C0303 trailing-whitespace error and removed them to fix it.

2.4 config.py

Errors raised by PyLint:

config.py:1:0: C0114: Missing module docstring

Iteration	Fix Applied
11	Added module docstring: "Contains all the constants used in the game".

2.5 dice.py

Errors raised by PyLint:

dice.py:1:0: C0114: Missing module docstring | dice.py:2:0: E0401: Unable to import 'moneypoly.config' | dice.py:25:12: W0201: Attribute 'doubles_streak' defined outside __init__ | dice.py:2:0: W0611: Unused BOARD_SIZE imported from moneypoly.config

Iteration	Fix Applied
12	Removed unused BOARD_SIZE imported from moneypoly.config.
13	Added module docstring: "Simulates rolling two six-sided dice and tracks consecutive doubles."
14	Defined self.doubles_streak inside __init__ (it was previously defined outside __init__).

2.6 game.py

Errors raised by PyLint:

game.py:453:0: C0325: Unnecessary parens after 'not' | game.py:462:0: C0325: Unnecessary parens after 'not' | game.py:468:0: C0304: Final newline missing | game.py:1:0: C0114: Missing module docstring | game.py:3:0: E0401: Unable to import 'moneypoly.config' | game.py:12:0: E0401: Unable to import 'moneypoly.player' | game.py:13:0: E0401: Unable to import 'moneypoly.board' | game.py:14:0: E0401: Unable to import 'moneypoly.bank' | game.py:15:0: E0401: Unable to import 'moneypoly.dice' | game.py:16:0: E0401: Unable to import 'moneypoly.cards' | game.py:17:0: E0401: Unable to import 'moneypoly' | game.py:20:0: R0902: Too many instance attributes (9/7) | game.py:295:4: R0912: Too many branches (15/12) | game.py:381:28: W1309: f-string without interpolation | game.py:402:12: R1723: Unnecessary

elif after break | game.py:1:0: W0611: Unused import os | game.py:3:0: W0611: Unused GO_TO_JAIL_POSITION imported

Iteration	Fix Applied
15	Fixed all import errors — changed all 'import moneypoly.module' statements to direct module imports.
16	Added module docstring: "Main game logic and flow control for MoneyPoly." Removed unused import os.
17	Added missing final newline at the end of the module.
18	Removed extra brackets in if statements: 'if not (0 <= idx < len(others))' to 'if not 0 <= idx < len(others)' and 'if not (0 <= pidx < len(player.properties))' to 'if not 0 <= pidx < len(player.properties)'.
19	Removed unused GO_TO_JAIL_POSITION import from config.
20	Removed non-required f-string: 'ui.print_banner(f"GAME OVER")' changed to 'ui.print_banner("GAME OVER")'.
21	Changed elif to if after break: 'if choice == 0: break / elif choice == 1:' changed to 'if choice == 0: break / if choice == 1:' (elif not required after break).
22	Resolved too-many-branches by combining two branches with the same action: merged 'elif action == "birthday"' and duplicate into 'elif action == ("birthday", "collect_from_all")' with shared logic block.
23	Resolved too-many-instance-attributes by grouping related fields into self.state = {"current_index": 0, "turn_number": 0, "running": True} and self.decks = {"chance": CardDeck(CHANCE_CARDS), "community": CardDeck(COMMUNITY_CHEST_CARDS)}.

2.7 player.py

Errors raised by PyLint:

player.py:87:0: C0304: Final newline missing | player.py:1:0: C0114: Missing module docstring | player.py:2:0: E0401: Unable to import 'moneypoly.config' | player.py:5:0: R0902: Too many instance attributes (8/7) | player.py:45:8: W0612: Unused variable 'old_position' | player.py:1:0: W0611: Unused import sys

Iteration	Fix Applied
24	Added a final newline and a module docstring: "Defines the Player class, which represents a single player in a MoneyPoly game." Also removed unused import sys.
25	Fixed import error — changed 'from moneypoly.config import' to 'from config import'.
26	Removed the unused variable: 'old_position = self.position'.
27	Resolved too-many-instance-attributes by grouping related attributes.

2.8 property.py

Errors raised by PyLint:

property.py:1:0: C0114: Missing module docstring | property.py:1:0: R0902: Too many instance attributes (9/7) | property.py:6:4: R0913: Too many arguments (6/5) | property.py:6:4: R0917: Too many positional arguments (6/5) | property.py:48:8: R1705: Unnecessary else after return | property.py:64:0: C0115: Missing class docstring for PropertyGroup

Iteration	Fix Applied
28	Added class docstring for PropertyGroup: "Represents a colour group of properties on the board." Added module docstring: "Defines the Property and PropertyGroup classes, which represent the purchasable properties on the board."
29	Removed unnecessary else as the if block ends with return 0.
30	Fixed too-many-arguments by making a pricing tuple: 'Property("Mediterranean Avenue", 1, 60, 2, g["brown"])' changed to 'Property("Mediterranean Avenue", 1, (60, 2), g["brown"])'.
31	Fixed too-many-instance-attributes by grouping owner, is_mortgaged, and houses: self.state = {"owner": None, "is_mortgaged": False, "houses": 0}.

2.9 ui.py

Errors raised by PyLint:

ui.py:1:0: C0114: Missing module docstring | ui.py:69:4: W0702: No exception type(s) specified (bare-except)

Iteration	Fix Applied
32	Added module docstring: "Functions for displaying game information and prompting user input."
33	Specified 'except ValueError' as it is the only possible error (replaced bare except).

2.10 main.py

Errors raised by PyLint:

main.py:1:0: C0114: Missing module docstring | main.py:4:0: C0116: Missing function docstring | main.py:11:0: C0116: Missing function docstring

Iteration	Fix Applied
34	Added module docstring and function docstrings for both get_player_names() and main().

3. Whitebox Test Suite (Q1.3)

All tests are written using PyTest. Rows shaded in grey are dedicated bug-detection tests — written specifically to expose a logic error found during whitebox analysis. These tests are expected to fail until the identified bug is fixed.

3.1 test_bank.py

1. TestGetBalance

Test	Description
`test_initial_balance_is_correct`	Checking if the starting funds are correct.
`test_balance_reflects_changes`	Checking if the balance is updating correctly.

2. TestCollect

Test	Description
`test_collect_positive`	Checking the normal case — collecting increases the bank funds.
`test_collect_zero`	Edge case — balance remains unchanged when zero is collected.
`test_collect_negative`	BUG TEST — docstring says negative amounts are silently ignored but code does self._funds += amount which would reduce the funds.
`test_collect_large_amount`	Check for edge case of very large amount.

3. TestPayOut

Test	Description
`test_pay_out_normal`	If the bank has sufficient funds the amount requested is returned and the bank funds are reduced.
`test_pay_out_zero`	Amount requested is 0, so 0 returned and the bank funds remain unchanged.
`test_pay_out_negative`	Amount requested is negative, so 0 returned and the bank funds remain unchanged.
`test_pay_out_exactly_all_funds`	Edge case where the amount requested is exactly equal to the bank funds, so the amount is returned and the bank funds become 0.
`test_pay_out_insufficient_funds_raises`	Amount > funds, so ValueError raised.
`test_pay_out_one_over_balance_raises`	Edge case — amount requested exactly one dollar over balance raises ValueError.

4. TestGiveLoan

Test	Description
`test_give_loan_normal`	BUG TEST — player balance increases by loan amount. Docstring says bank funds are reduced accordingly but code never does.
`test_give_loan_records`	Loan recorded with loan_count incrementing and total_loans_issued() adding the loan amount.
`test_give_loan_zero`	Loan amount is 0 — nothing happens.
`test_give_loan_negative`	Loan has negative amount — nothing happens.
`test_give_loan_multiple_loans`	Testing for multiple loans all tracked correctly.

5. TestTotalLoansIssued

Test	Description
`test_no_loans`	No loans issued — total is 0.
`test_single_loan`	Normal case — single loan recorded correctly.
`test_multiple_loans`	Multiple loans summed correctly.

6. TestLoanCount

Test	Description
`test_no_loans`	Edge case — no loans issued, count is 0.
`test_increments`	Testing if count increases with each loan.

3.2 test_board.py

1. TestGetPropertyAt

Test	Description
`test_returns_property`	Checking normal case where a property exists at a valid position; verifies correct property object and name returned.
`test_returns_none_for_Go`	Edge case where position corresponds to a special tile (Go), so no property exists and function returns None.
`test_returns_none_for_nonexistent`	Invalid position off the board, ensuring function safely returns None.
`test_returns_boardwalk`	Checking correct property retrieval at a high valid index; ensures correct mapping at board boundary.
`test_returns_none_for_jail`	Special tile case (Jail), ensuring no property is returned.
`test_returns_none_for_chance`	Special tile case (Chance), ensuring no property is returned.

2. TestGetTileType

Test	Description
`test_position_go`	Position exists in SPECIAL_TILES; verifies correct identification of "go".
`test_jail_position`	Position exists in SPECIAL_TILES; verifies "jail".
`test_go_to_jail`	Position exists in SPECIAL_TILES; verifies "go_to_jail".
`test_free_parking`	Position exists in SPECIAL_TILES; verifies "free_parking".
`test_income_tax`	Position exists in SPECIAL_TILES; verifies "income_tax".
`test_luxury_tax`	Position exists in SPECIAL_TILES; verifies "luxury_tax".
`test_chance`	Position exists in SPECIAL_TILES; verifies "chance".
`test_community_chest`	Position exists in SPECIAL_TILES; verifies "community_chest".
`test_railroad`	Position exists in SPECIAL_TILES; verifies "railroad".
`test_property`	Position not in SPECIAL_TILES but has a property; ensures it is classified as "property".
`test_blank`	Position not in SPECIAL_TILES and no property exists; ensures it returns "blank".
`test_all_railroads`	Iterates over all railroad positions to ensure consistency across all such tiles.
`test_all_chance`	Iterates over all chance positions to ensure consistency.
`test_all_community_chest`	Iterates over all community chest positions to ensure consistency.

3. TestIsPurchasable

Test	Description
`test_unowned_not_mortgaged_is_purchasable`	Normal case where property has no owner and is not mortgaged; should return True.
`test_special_tile`	Edge case where position corresponds to a special tile; no property exists so returns False.
`test_invalid_position`	Invalid/off-board position; returns False.
`test_owned_property`	Property already has an owner; should not be purchasable.
`test_mortgaged_property`	Property is mortgaged; should not be purchasable.
`test_boardwalk`	Valid high-value property that is initially purchasable; ensures correct default state.
`test_railroad`	Railroad treated as special tile; ensures it is not purchasable.

4. TestIsSpecialTile

Test	Description
`test_go`	Position exists in SPECIAL_TILES; returns True.
`test_jail`	Position exists in SPECIAL_TILES; returns True.

Test	Description
`test_property`	Position corresponds to a property; not in SPECIAL_TILES so returns False.
`test_invalid_position`	Edge case where position is off-board; should return False.
`test_all_special_tiles`	Iterates through all SPECIAL_TILES keys to ensure all are correctly identified as special tiles.

5. TestPropertiesOwnedBy

Test	Description
`test_no_properties_owned`	Player owns nothing; function should return empty list.
`test_returns_owned_properties`	Player owns multiple properties; ensures all owned properties are returned.
`test_returns_correct_players_properties`	Edge case with multiple players; ensures each player only gets their own properties.

6. TestUnownedProperties

Test	Description
`test_all_unowned_at_start`	Initial state where no properties are owned; all properties should be returned.
`test_owned_properties_excluded`	Owned properties should be filtered out from the result.
`test_all_owned`	Edge case where all properties are owned; function should return empty list.

7. TestBoardInit

Test	Description
`test_board`	Verifies total number of properties initialised on the board is correct (22).
`test_board_groups`	Verifies total number of color groups is correct (8).
`test_all_groups`	Ensures all expected group names are present in the board configuration.

3.3 test_cards.py

1. TestCardDeckInit

Test	Description
`test_chance`	Checking that the Chance deck initialises with exactly 12 cards.
`test_community`	Checking that the Community Chest deck initialises with exactly 12 cards.
`test_index`	Checking that the deck index starts at 0 upon initialisation.

Test	Description
`test_cards_are_copied`	Checking that the cards list is copied internally so modifying the deck does not affect the original list.
`test_empty_deck_initializes`	Edge case: initialising with an empty list should not crash, length is 0 and index is 0.

2. TestDraw

Test	Description
`test_draw`	Checking that the first draw returns the first card in the deck.
`test_draw_inc`	Checking that the index increments correctly after each draw.
`test_draw_order`	Checking that cards are drawn in the same order as they were added.
`test_draw_cycles`	Edge case: after all cards are drawn, the deck cycles back to the beginning.
`test_draw_empty_deck`	Edge case: drawing from an empty deck returns None.
`test_draw_empty_deck_index`	Checking that drawing from an empty deck does not change the index.
`test_draw_single_card_deck`	Edge case: a single-card deck always returns the same card on every draw.
`test_drawn_action`	Checking that a drawn card contains required fields: action, value, description.
`test_draw_all_actions_valid`	Checking that all Chance cards have valid action types.
`test_draw_community_all_actions_valid`	Checking that all Community Chest cards have valid action types.

3. TestPeek

Test	Description
`test_peek`	Checking that peek returns the next card without drawing it.
`test_peekindex`	Checking that peek does not modify the index even after multiple calls.
`test_peek_matches_next_draw`	Checking that the peeked card matches the next drawn card.
`test_peek_after_draws`	Checking that peek works correctly after some cards have already been drawn.
`test_peek_empty_deck`	Edge case: peeking an empty deck returns None.
`test_peek_cycles`	Checking that peek works correctly even after the deck cycles.

4. TestReshuffle

Test	Description
`test_reshuffle`	Checking that reshuffling resets the index back to 0.

Test	Description
`test_reshuffle_card_count`	Checking that reshuffling does not change the number of cards in the deck.
`test_reshuffle_drawable`	Checking that cards can still be drawn after reshuffling.
`test_reshuffle_empty_deck`	Edge case: reshuffling an empty deck does not crash and keeps index and length unchanged.

5. TestCardsRemaining

Test	Description
`test_full_deck`	Checking that a fresh deck reports all 12 cards remaining.
`test_decreases_draw`	Checking that the remaining card count decreases after each draw.
`test_resets_full_cycle`	Checking that after drawing all cards, remaining resets to full deck size.
`test_empty_deck_remaining`	BUG TEST — calling cards_remaining on empty deck raises ZeroDivisionError.

6. TestLen

Test	Description
`test_len_chance`	Checking that length of Chance deck is 12.
`test_len_community`	Checking that length of Community Chest deck is 12.
`test_len_empty`	Edge case: length of empty deck is 0.
`test_len_draw`	Checking that drawing cards does not change the total deck length.

7. TestCardValues

Test	Description
`test_chance_collect`	Checking that all Chance cards with "collect" action have positive values.
`test_chance_pay`	Checking that all Chance cards with "pay" action have positive values.
`test_community_collect`	Checking that all Community Chest "collect" cards have positive values.
`test_all_cards_keys`	Checking that every card contains required keys: description, action, value.
`test_move_to_boardwalk`	Checking that the "Advance to Boardwalk" card has value 39.
`test_move_to_go`	Checking that the "Advance to Go" card has value 0.

3.4 test_dice.py

1. TestDiceInit

Test	Description
`test_initial_zero`	Checking that both dice values start at 0 before any roll.
`test_initial_streak`	Checking that the doubles streak counter starts at 0 before any roll.

2. TestReset

Test	Description
`test_reset`	Checking that after reset both dice values are set back to 0.
`test_reset_streak`	Checking that after reset the doubles streak is reset to 0.

3. TestRoll

Test	Description
`test_roll`	Checking that roll() returns an integer value.
`test_roll_total`	Checking that the return value of roll() equals die1 + die2.
`test_roll_range`	BUG TEST — dice should generate values from 1 to 6, but current implementation uses randint(1,5) so 6 never appears.
`test_roll_minimum`	Checking that the minimum possible roll value is 2.
`test_roll_maximum`	BUG TEST — maximum roll should be 12 (6+6), but may never occur due to incorrect random range.
`test_roll_updates_die1`	Checking that after rolling, both dice values are within the valid range 1 to 6.
`test_roll_doubles_inc`	Checking that when doubles occur, the doubles streak increments correctly using mocked values.
`test_roll_consec_inc`	Checking that multiple consecutive doubles correctly increment the streak count.
`test_roll_doubles_resets`	Checking that when a non-double roll occurs, the doubles streak resets to 0.
`test_roll_streak_resets_after_non_doubles`	Checking that after a streak of doubles, a non-double resets the streak to 0 instead of decrementing.

4. TestIsDoubles

Test	Description
`test_is_doubles`	Checking that when both dice have equal values, is_doubles() returns True.
`test_is_doubles_false`	Checking that when dice values differ, is_doubles() returns False.
`test_is_doubles_all_values`	Checking that all valid double combinations from (1,1) to (6,6) return True.

5. TestTotal

Test	Description
`test_total`	Checking that total() correctly returns the sum of die1 and die2.
`test_total_minimum`	Checking that the minimum total value is 2.
`test_total_maximum`	Checking that the maximum total value is 12.
`test_total_zero_before_roll`	Checking that total() returns 0 before any roll has occurred.

6. TestDescribe

Test	Description
`test_describe`	Checking that describe() output includes both dice values and their total.
`test_describe_doubles_note`	Checking that when doubles occur, the description includes a "DOUBLES" note.
`test_describe_no_note`	Checking that when not doubles, the description does not include "DOUBLES".
`test_describe_format`	Checking that the output format correctly displays the expression "die1 + die2 = total".

3.5 test_game.py

1. TestBuyProperty

Test	Description
`test_insufficient_funds`	Balance < price, returns False.
`test_exact`	BUG TEST — balance == price; player with exactly enough money should be able to buy.
`test_more`	Balance > price, purchase succeeds.
`test_sets_owner`	Purchase assigns property owner correctly.
`test_deducts`	Purchase reduces player balance by exact property price.
`test_add_to_prop`	Property is successfully added to player's property list.
`test_failed_buy`	Failed purchase leaves property owner as None.
`test_failed_balance`	Failed purchase leaves player balance unchanged.

2. TestPayRent

Test	Description
`test_mortgaged`	Property is mortgaged; rent is not paid and renter balance remains unchanged.
`test_no_owner_no_rent_charged`	Owner is None; returns immediately and balance is unchanged.

Test	Description
`test_rent_deducted_from_renter`	Normal case; renter's balance is reduced by the rent amount.
`test_rent_transferred_to_owner`	BUG TEST — owner must receive the money deducted from the renter.

3. TestFindWinner

Test	Description
`test_no_players_returns_none`	Empty players list returns None.
`test_single_player_returns_that_player`	Only one player in game, that player is returned as winner.
`test_returns_richest_player`	BUG TEST — in a game with multiple players, the one with the highest balance wins.
`test_winner_with_equal_balance`	Edge case; if balances are equal, any of the richest players is acceptable.

4. TestMortgageProperty

Test	Description
`test_not_owner_returns_false`	Player does not own the property, returns False.
`test_already_mortgaged_returns_false`	Property is already mortgaged, returns False.
`test_successful_mortgage_returns_true`	Valid mortgage returns True.
`test_successful_mortgage_credits_player`	Player balance increases by the property's mortgage value.
`test_successful_mortgage_sets_flag`	Property status is_mortgaged is set to True.

5. TestUnmortgageProperty

Test	Description
`test_not_owner_returns_false`	Player does not own the property, returns False.
`test_not_mortgaged_returns_false`	Property is not currently mortgaged, returns False.
`test_cannot_afford_returns_false`	Player cannot afford the 10% interest redemption cost, returns False.
`test_successful_unmortgage_returns_true`	All checks pass, returns True.
`test_successful_unmortgage_deducts_balance`	Redemption cost (mortgage value * 1.1) is deducted from player.

Test	Description
`test_successful_unmortgage_clears_flag`	Property status is_mortgaged is set to False.

6. TestTrade

Test	Description
`test_seller_not_owner_fails`	Seller does not own property, returns False.
`test_buyer_cannot_afford_fails`	Buyer has insufficient funds for the trade price, returns False.
`test_successful_trade_returns_true`	Valid trade returns True.
`test_successful_trade_changes_owner`	Property ownership is transferred to the buyer.
`test_successful_trade_deducts_buyer_balance`	The trade price is deducted from the buyer's balance.
`test_successful_trade_moves_property_list`	Property is removed from seller's list and added to buyer's.
`test_zero_cash_trade_succeeds`	Edge case; trade for $0 is valid and succeeds.

7. TestCheckBankruptcy

Test	Description
`test_positive_balance_not_eliminated`	Player balance > 0, player remains in the game.
`test_zero_balance_eliminated`	Player balance == 0, player is eliminated from the game.
`test_negative_balance_eliminated`	Player balance < 0, player is eliminated.
`test_bankruptcy_releases_properties`	Properties owned by the bankrupt player are released back to the bank.
`test_bankruptcy_clears_property_list`	The bankrupt player's property list is cleared.
`test_bankruptcy_sets_eliminated_flag`	The player's is_eliminated flag is set to True.

8. TestAdvanceTurn

Test	Description
`test_turn_number_increments`	Turn number increases and moves current player index to the next person.
`test_index_wraps_to_zero`	After the last player, the index wraps back to the first player.

9. TestCurrentPlayer

Test	Description
`test_first_player_is_alice`	Initial current player is the first person in the player list.
`test_after_advance_current_is_bob`	Current player updates correctly after the turn is advanced.

3.6 test_player.py

1. TestInit

Test	Description
`test_start_balance`	Player starts with the default configured starting balance.
`test_custom_balance`	Player can be initialised with a specific custom balance.
`test_start_pos`	Player starts the game at position 0.
`test_start_prop`	Player starts with an empty list of properties.
`test_not_eliminated`	Player starts with the is_eliminated flag set to False.
`test_jail_dict`	Jail dictionary is initialised with correct default values (not in jail, 0 turns, 0 cards).

2. TestAddMoney

Test	Description
`test_add_pos`	Positive amounts correctly increase the player's balance.
`test_add_zero`	Adding zero leaves the balance unchanged.
`test_add_neg`	Negative amounts must raise a ValueError.
`test_add_neg_msg`	The error message for negative additions must include the invalid amount.
`test_add_large_amount`	Adding very large sums is handled correctly.
`test_multiple`	Multiple additions successfully accumulate in the balance.

3. TestDeductMoney

Test	Description
`test_deduct_pos`	Positive amounts correctly reduce the player's balance.
`test_deduct_zero`	Deducting zero leaves the balance unchanged.
`test_deduct_neg`	Negative amounts must raise a ValueError.
`test_deduct_entire_balance`	Edge case; deducting the exact balance amount leaves the player at 0.
`test_deduct_more_than_balance`	Balance can drop below zero (indicating bankruptcy).
`test_deduct_large_amount`	Large deductions are handled correctly.

4. TestIsBankrupt

Test	Description
`test_not_bankrupt`	Balance > 0 returns False.
`test_bankrupt_zero`	Balance of exactly 0 returns True.
`test_bankrupt_neg`	Balance < 0 returns True.
`test_bankrupt_1`	Balance of exactly 1 returns False.
`test_becomes_bankrupt_after_deduction`	Player correctly shifts to bankrupt status after losing all money.

5. TestMove

Test	Description
`test_normal_move`	Simple movement within the board boundaries.
`test_move_no_go_salary`	Moving without landing on or passing Go does not award a salary.
`test_move_on_go`	Landing exactly on position 0 awards the GO salary.
`test_move_passing_go`	BUG TEST — passing position 0 without landing on it must still award the GO salary.
`test_move_wraparound`	Player position wraps correctly when passing the board size limit.
`test_move_returns_new_pos`	The move() method returns the resulting new position.
`test_move_from_zero`	Moving away from the Go position works correctly.

6. TestNetWorth

Test	Description
`test_net_worth`	If no properties are owned, net worth is equal to the current balance.
`test_net_worth_includes_properties`	BUG TEST — property values (mortgage value) must be included in total net worth.
`test_net_worth_multiple_properties`	BUG TEST — values of all owned properties must be summed together.
`test_net_worth_after_balance_change`	Net worth reflects changes to the liquid cash balance.

7. TestGoToJail

Test	Description
`test_go_to_jail`	Player is moved directly to the JAIL_POSITION.
`test_go_to_jail_flag`	The in_jail flag in the jail dictionary is set to True.
`test_go_to_jail_resets_jail_turns`	Existing jail turn counts are reset to 0 upon entering jail.

Test	Description
`test_go_to_jail_from_any_position`	Movement to jail works regardless of starting position.
`test_go_to_jail_does_not_affect_balance`	Entering jail does not change the player's balance.

8. TestAddProperty

Test	Description
`test_add_property_append`	A property is successfully added to the player's holdings.
`test_add_property_no_duplicates`	If a property is already owned, it is not added to the list again.
`test_add_multiple_properties`	Multiple distinct properties are all tracked correctly.

9. TestRemoveProperty

Test	Description
`test_remove_existing_property`	An owned property is successfully removed from the player's list.
`test_remove_nonexistent_prop`	Attempting to remove a property the player doesn't own does nothing.
`test_remove_leaves_others`	Removing one property does not affect other owned properties.

10. TestCountProperties

Test	Description
`test_count_zero_at_start`	Initial property count is 0.
`test_count_after_adding`	Property count accurately reflects the number of items added.
`test_count_after_removing`	Property count decreases correctly after a property is removed.

11. TestStatusLine

Test	Description
`test_status_line_not_jailed`	The status output does not include a [JAILED] tag for free players.
`test_status_line_jailed`	The [JAILED] tag appears in the output when the player is imprisoned.
`test_status_line_contains_name`	Status line includes the player's name.

Test	Description
`test_status_line_contains_balance`	Status line includes the current balance.
`test_status_line_contains_position`	Status line includes the current position.

3.7 test_property.py

1. TestPropertyInit

Test	Description
`test_price_set_from_pricing_tuple`	Verify the property price is correctly pulled from the first element of the pricing tuple.
`test_base_rent_set_from_pricing_tuple`	Verify the base rent is correctly pulled from the second element of the pricing tuple.
`test_mortgage_value_is_half_price`	Mortgage value is calculated as exactly half of the property price.
`test_mortgage_value_floors_odd_price`	Edge case; verify that odd property prices floor correctly when calculating mortgage value.
`test_state_owner_starts_none`	Property owner must be initialised as None.
`test_state_mortgaged_starts_false`	Property must start in an unmortgaged state.
`test_state_houses_starts_zero`	Property starts with a house count of 0.
`test_registered_with_group_on_creation`	Verify property automatically registers itself with its assigned PropertyGroup during initialisation.
`test_not_registered_twice_in_group`	Edge case; ensure a property is not duplicated within its group's list.
`test_no_group_is_fine`	Property initialisation succeeds even if no group is assigned.

2. TestGetRent

Test	Description
`test_mortgaged_returns_zero`	If property is mortgaged, rent returned must be 0.
`test_no_group_returns_base_rent`	Properties without a group return their base rent directly.
`test_partial_group_returns_base_rent`	BUG TEST — if a player only owns part of a group, rent should remain at the base amount (not double).
`test_full_group_doubles_rent`	If a player owns all properties in a group, the rent for those properties is doubled.
`test_unowned_property_returns_base_rent`	Properties with no owner return the base rent amount.
`test_multiplier_is_two`	Edge case; verify the constant FULL_GROUP_MULTIPLIER is set to 2.

Test	Description
`test_three_property_group_all_owned`	Edge case; verify rent doubling works correctly for groups containing three properties.
`test_three_property_group_partial_ownership`	Edge case; verify rent does NOT double if a player owns only 2 out of 3 properties in a group.

3. TestMortgage

Test	Description
`test_mortgage_returns_half_price`	Mortgaging a property returns a payout equal to half its price.
`test_mortgage_sets_flag`	Property status is_mortgaged is set to True after calling mortgage().
`test_mortgage_already_mortgaged_returns_zero`	Calling mortgage on an already mortgaged property returns a 0 payout.
`test_mortgage_already_mortgaged_stays_mortgaged`	Edge case; calling mortgage on a mortgaged property does not toggle the status back.
`test_mortgage_cheap_property`	Edge case; verify floor division for payouts on low-cost properties.
`test_mortgage_expensive_property`	Edge case; verify payouts for high-cost properties.

4. TestUnmortgage

Test	Description
`test_unmortgage_not_mortgaged_returns_zero`	Calling unmortgage on an unmortgaged property results in a 0 cost.
`test_unmortgage_returns_correct_cost`	Unmortgaging costs the mortgage value plus 10% interest.
`test_unmortgage_clears_flag`	Property status is_mortgaged is set to False after unmortgaging.
`test_unmortgage_boardwalk_cost`	Edge case; verify specific cost calculation for high-end property unmortgaging.
`test_unmortgage_cheap_property`	Edge case; verify interest calculation for low-cost property unmortgaging.
`test_unmortgage_not_mortgaged_flag_unchanged`	Edge case; unmortgaging a clear property keeps the flag False.

5. TestIsAvailable

Test	Description
`test_unowned_not_mortgaged_is_available`	Property is available if it has no owner and is not mortgaged.
`test_owned_not_available`	Property is unavailable if it already has an owner.

Test	Description
`test_mortgaged_not_available`	Property is unavailable if it is currently mortgaged.
`test_owned_and_mortgaged_not_available`	Edge case; property is unavailable if both owned and mortgaged.

6. TestPropertyGroup

Test	Description
`test_group_starts_empty`	A new property group contains no properties by default.
`test_add_property_adds_to_group`	add_property successfully appends a property to the group list.
`test_add_property_sets_back_link`	Adding a property to a group correctly updates the property's group attribute.
`test_add_property_not_duplicated`	Ensure properties are not added to the same group more than once.
`test_size_empty_group`	Edge case; an empty group reports a size of 0.
`test_size_with_properties`	Group size correctly reflects the number of registered properties.

7. TestAllOwnedBy

Test	Description
`test_player_none_returns_false`	If player is None, returns False immediately.
`test_empty_group`	An empty group cannot be "owned" by anyone, returns False.
`test_single_property_owned_returns_true`	Group with one property returns True if the specified player owns it.
`test_single_property_not_owned_returns_false`	Group with one property returns False if the player does not own it.
`test_partial_ownership_returns_false`	BUG TEST — if a player owns some but not all properties in a group, returns False.
`test_full_ownership_returns_true`	Returns True only if the specified player owns every property in the group.
`test_different_player_owns_all_returns_false`	Edge case; returns False if the properties are owned by someone other than the target player.

8. TestGetOwnerCounts

Test	Description
`test_no_owners_returns_empty_dict`	Returns an empty dictionary if no properties in the group are owned.
`test_one_owner_counted`	Correctly identifies and counts properties for a single owner.
`test_one_player_owns_multiple`	Edge case; correctly tallies multiple properties owned by the same player.

Test	Description
`test_two_players_each_own_one`	Edge case; split ownership between players is tracked and counted accurately.

3.8 test_ui.py

1. TestSafeIntInput

Test	Description
`test_valid_integer_returns_int`	Normal case; a valid integer string is successfully converted and returned as an int.
`test_invalid_string_returns_default`	A string containing letters raises a ValueError and returns the provided default value.
`test_empty_input_returns_default`	Edge case; an empty string input returns the default value.
`test_negative_integer`	Edge case; negative numbers are correctly parsed as valid integers.
`test_zero`	Edge case; the number zero is correctly parsed as a valid integer.
`test_float_string_returns_default`	Edge case; a string representing a decimal (float) returns the default value.
`test_symbols_return_default`	Edge case; strings containing currency symbols or other characters return the default value.
`test_large_integer_valid`	Edge case; very large integers are parsed correctly without error.

2. TestConfirm

Test	Description
`test_y_returns_true`	User types 'y', confirming the action and returning True.
`test_n_returns_false`	User types 'n', declining the action and returning False.
`test_uppercase_y_returns_true`	Input is case-insensitive; 'Y' returns True.
`test_uppercase_n_returns_false`	Input is case-insensitive; 'N' returns False.
`test_empty_input_returns_false`	Pressing Enter with no input defaults to False.
`test_random_word_returns_false`	Any input other than 'y' (like "yes") results in False.
`test_spaces_stripped_y_returns_true`	Leading or trailing whitespace is removed, so ' y ' returns True.
`test_spaces_stripped_n_returns_false`	Whitespace is removed, so ' n ' returns False.

3. TestFormatCurrency

Test	Description
`test_formats_basic_amount`	Standard amounts are formatted with a dollar sign and appropriate commas.
`test_formats_zero`	Zero is formatted correctly as "$0".
`test_formats_large_amount`	Large numbers (millions) include multiple comma separators.
`test_formats_small_amount`	Amounts under 1,000 are formatted without commas.

4. Bug Summary

The following logical errors were discovered during whitebox testing. Each has a corresponding dedicated test case that will fail until the bug is resolved. Rows are shaded in grey.

#	Module	Bug Description	Test Case
1	`bank.py`	collect() adds negative amounts to funds instead of silently ignoring them.	`test_collect_negative`
2	`bank.py`	give_loan() increases player balance but the code never deducts the loan amount from bank funds.	`test_give_loan_normal`
3	`dice.py`	randint(1,5) is used instead of randint(1,6) — the die value 6 can never be rolled.	`test_roll_range`
4	`dice.py`	Maximum roll should be 12 (6+6), but cannot occur due to the incorrect random range.	`test_roll_maximum`
5	`game.py`	buy_property() rejects the purchase when player balance == price; it should succeed.	`test_exact`
6	`game.py`	pay_rent() deducts from the renter but does not credit the money to the property owner.	`test_rent_transferred_to_owner`
7	`game.py`	find_winner() does not correctly identify the player with the highest balance.	`test_returns_richest_player`
8	`player.py`	move() does not award the GO salary when the player passes position 0 without landing on it.	`test_move_passing_go`
9	`player.py`	net_worth() does not include owned property mortgage values in the total.	`test_net_worth_includes_properties`
10	`player.py`	net_worth() does not sum values across multiple owned properties.	`test_net_worth_multiple_properties`
11	`property.py`	get_rent() doubles rent when a player has partial group ownership; full ownership should be required.	`test_partial_group_returns_base_rent`
12	`property.py`	all_owned_by() returns True for partial group ownership; it should return False unless the player owns all.	`test_partial_ownership_returns_false`